Marisa Canizales- May 4th, 2026

COP 4834 Web Systems II

Assignment 14: Final Documentation

Professor Dr. Ron Eaglin

Table of Contents:

1. Project Overview and Summary:

Objectives:
The objective for this project was to design and develop a full stack web based CRUD application that allows users to manage user data. This system provides a secure login functionality and role based access control to help ensure proper security. The role based access lets only authorized users perform certain operations such as adding and deleting users and records. The main objectives that this application completes is it is able to implement a functional user login system, allows users to view and manage records, restricts certain actions based on specific user roles, and demonstrates real world application development practices.

Key Stakeholders and Roles:
Developer: The developer is responsible for designing, developing, and maintaining the application.

End Users: Any individuals who interact with the system to manager user data

Administrator: Users with the specific privileges to add, update, or delete records.

Instructors: Evaluate the application and the documentation

Business Value and Expected Outcomes:
This application provides the following values

- Reducing time required for users to manage data
- Provides a structured and secure way to store and edit user information
- Serves as a foundation for any possible future enterprise applications

Critical Success Factors:

- Successful user authentication
- Functional CRUD
- Proper enforcement for the role based access control
- User friendly interface
- Minimal errors during usage

2. Client- Side Architecture

The client side of this application is browser based and can be accessed through web browsers such as Chrome or Microsoft Edge.

Features:

- Login interface
- User Dashboard
- Forms to create and edit user information

Technologies Used:

- HTML
- CSS
- Javascript

Server-Side Architecture

The server side is built using Node.js with Express and handles all the requests of the client.

Responsibilities:

- Processing User authentication
- Handling different CRUD operations
- Communicating with the database and requests

Database Design:

This system uses a relational database to store the user information

Authentication and Authorization:

- Users must log in using specific credentials
- Role-based access control is implemented used different views based on which user is logged in

Hosting Requirements:
This application was created through Replit so it allows for cloud-based access and execution for users.

3. <u>Functional Requirements:</u>

User Story 1: User Login

- Actor: User
- Preconditions: User account exists
- Flow Events
    1. User enters username and password
    2. System validates or rejects credentials
    3. User is successfully logged in
- Postconditions: The user gains access to the system and information stored inside
- Exception: If the user has invalid credentials this will result in an error message

User Story 2: View Users

- Actor: User
- Flow Events
    1. User logs in to system
    2. Dashboard with valid users pops up
- Postconditions: The user is able to view every other registered user and their roles

User Story 3: Update User

- Actor: User
- Flow Events
    1. User logs in under admin credentials
    2. User is able to pick and edit the user information
- Postconditions: The user is able to update any current user information

User Story 4: Delete User

- Actor: User
- Flow Events
    1. User logs in under admin credentials
    2. User picks specific use and selects the trash can icon to delete the use
- Postconditions: The user is able to delete a specific user from a system and is not longer able to view them from the registered user list.

Business Rules:

- All email address/ usernames must be unique
- Unique passwords are required
- Admins are the only users that can create, update, or delete users
- Data must be valid before updating and submitting

4. Non-Functional Requirements

Performance Requirements:

- The application must load within 10 seconds
- The system should be able to handle multiple users simultaneously

Security Requirements

- Secure log in authentication
- Role-based authorization
- Protection against any SQL injection
- Can securely handle user data

Scalability and Availability:

- The system should support future database expansion
- Capable of handling an increased user load
- The backup and recovery strategies are implemented

5. Data Management

Data Models and Relationships:

Primary table: Users

- ID
- Name
- Email
- Password
- Role

Data Flow:

1. The user inputs the data
2. The data sent to the server via API
3. The server processes request
4. The database is now updated
5. The response returned to the user

Data Retention and Privacy:

- The user data is stored securely
- Users sensitive information is protected
- Data access is restricted based on roles

Backup Strategies:

- Database persistence is handled by Replit
- Data can be exported if necessary

6. Interface Specifications:

API Endpoints:

GET/users
- Retrieves all users

POST/users
- Creates a new user

PUT/users/:id
- Updates user information

DELETE/users/:id
- Deletes a user (only admin can only)

Error Handling:
- Invalid login returns an error message
- Failed updates return a system error
- Unauthorized actions are blocked

External Interfaces:
- No external APIs are integrated within this system

7. Infrastructure Requirements:

Development Environment: **Replit**

Testing Environment: **Web Browser testing**

Production Environment: **Replit deployment**

Deployment Requirements: **Cloud-based hosting**

8. Quality Assurance:

Testing Requirements:
- Unit testing for backend logic
- Manual testing is used for interface

Acceptance Criteria:
- Users can log in successfully
- CRUD operations can function correctly
- Admin restrictions are enforced to all users

Performance Metrics:
- System must have a fast response time
- No major errors appeared during testing

Security Testing:
- Will validate the login protection
- Will ensure unauthorized users cannot access restricted actions

9. Documentation Needs:

This system requires the following documentation:
- Technical documentation
- User documentation (directions how to use the app)
- API documentation
- Deployment instructions
- Maintenance documentation

10. Project Constraints and Assumptions:

Technical Constraints:
- Limited to the Replit environment
- Has a basic database implementation

Business Constraints:

- Developed for academic purposes
- Has a limited real-world deployment

Timeline Constraints:

- Project completed over multiple weekly assignments

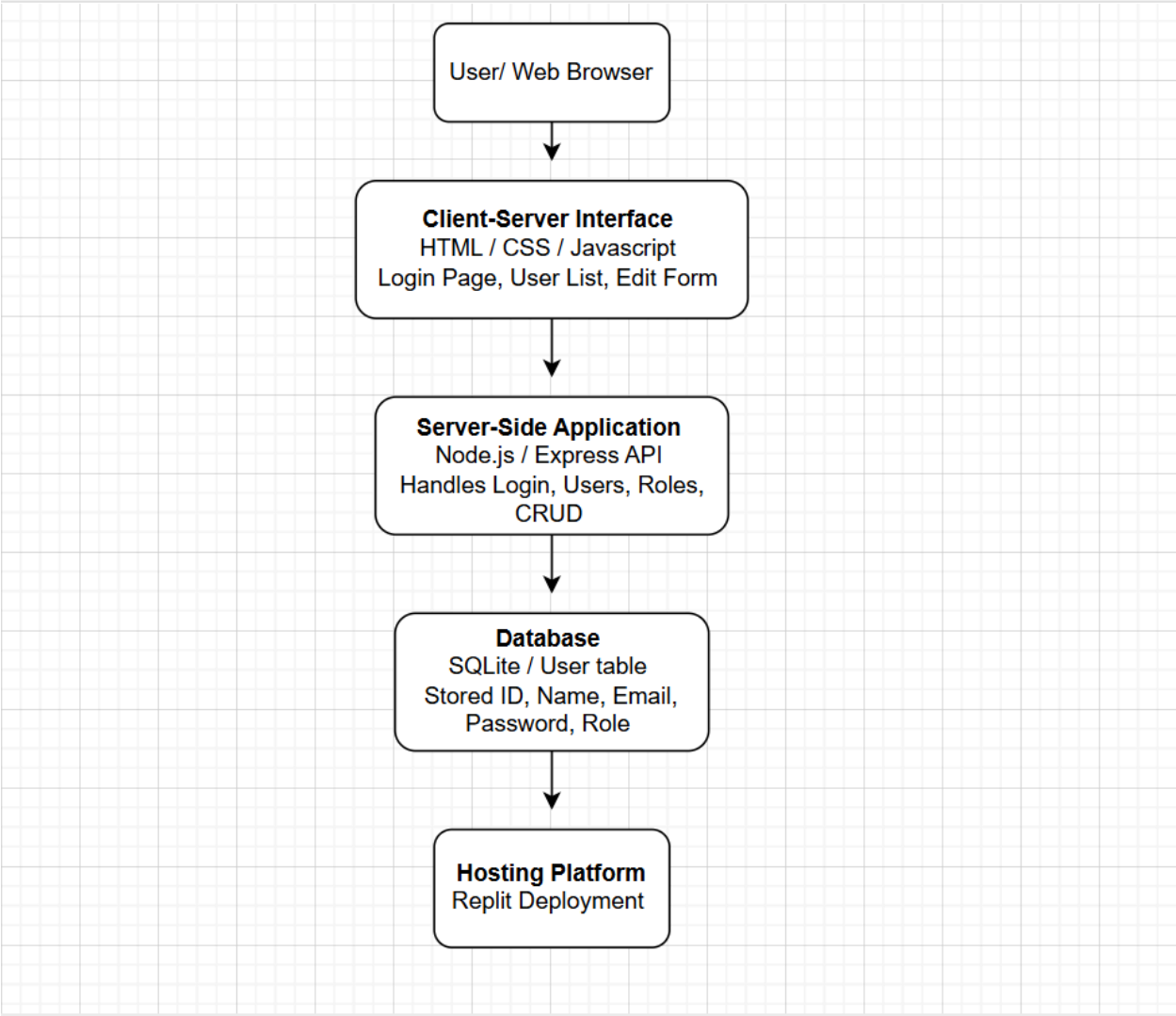
Resources Constraints:

- A single developer project

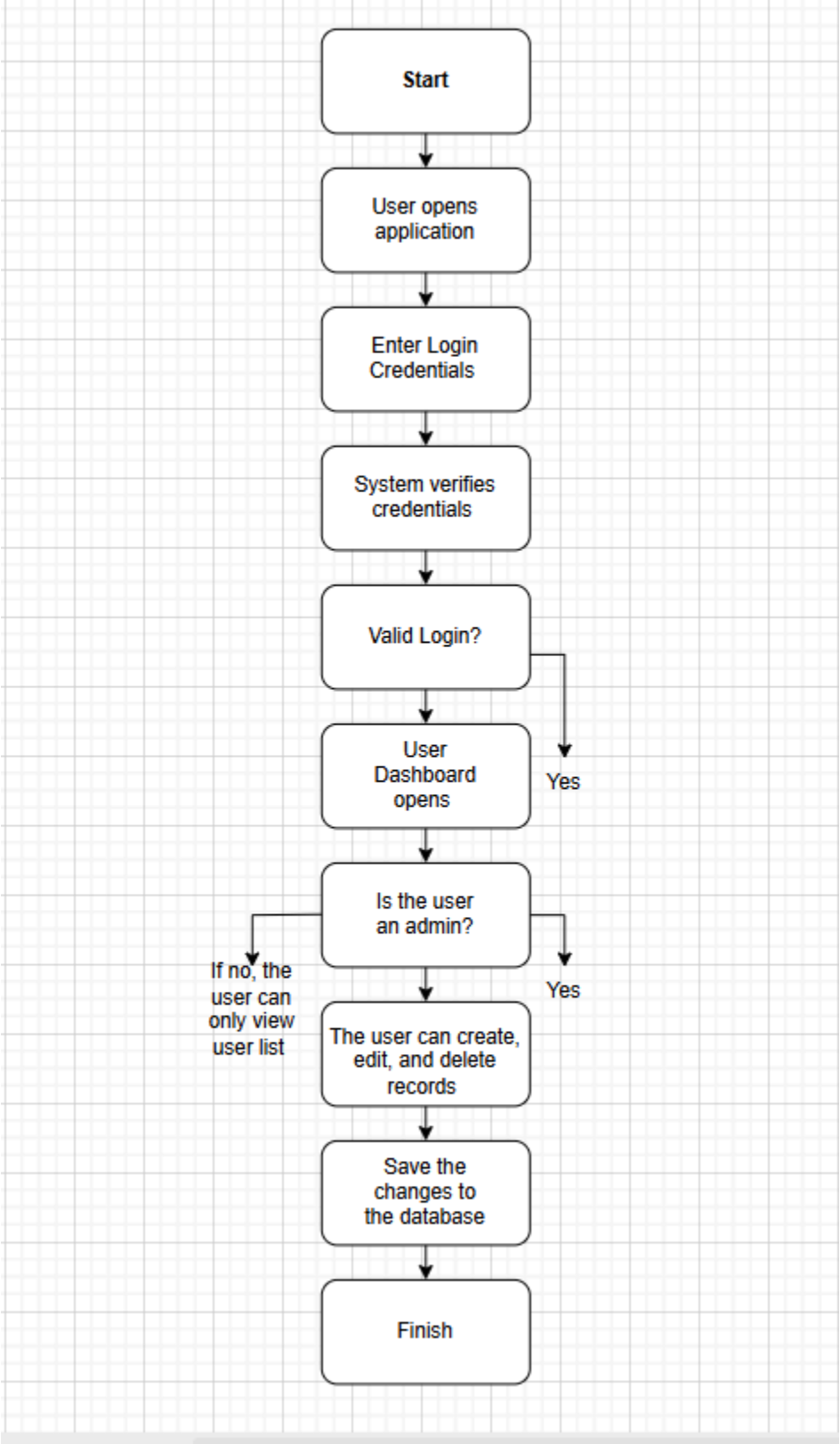
Project Dependencies:

- Node.js
- Express framework
- Database system

Appendix A: Architecture Diagram

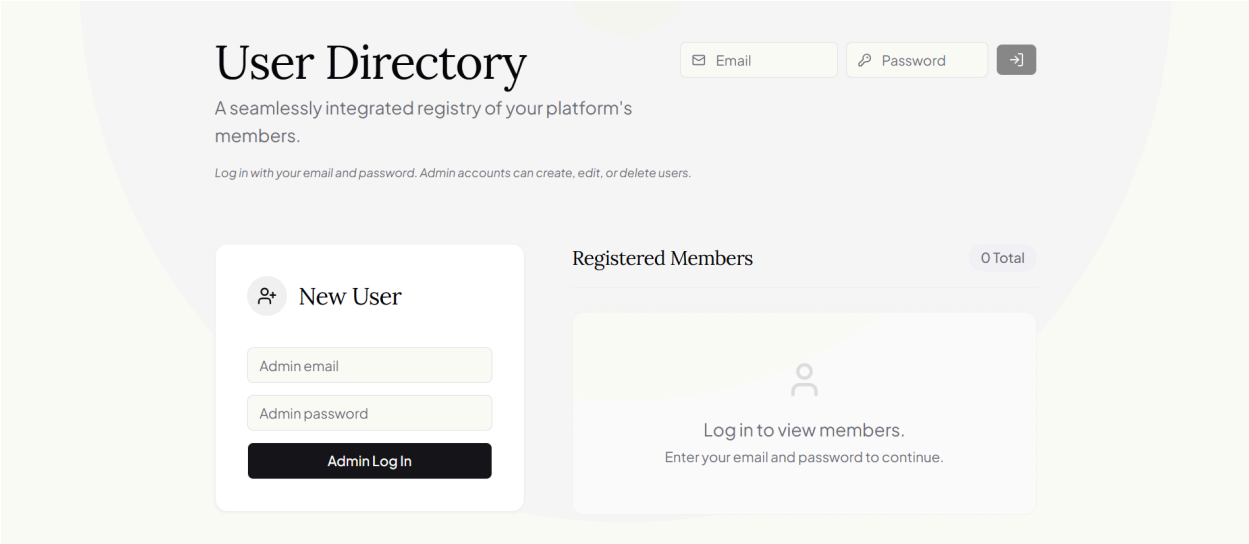


Appendix B: Workflow Diagram

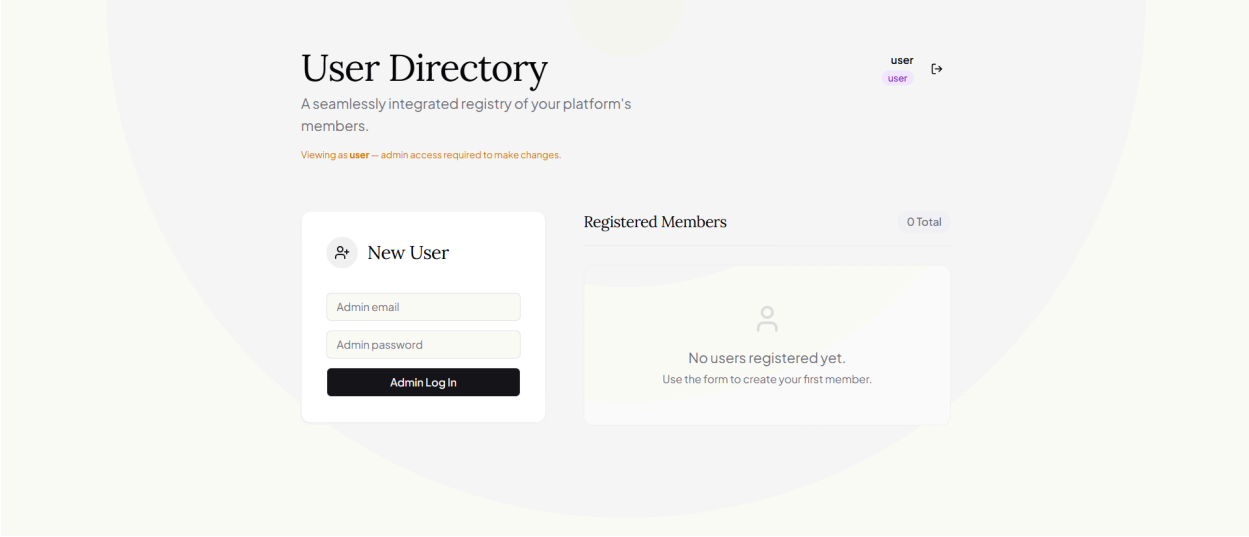


Appendix C: Screenshots:

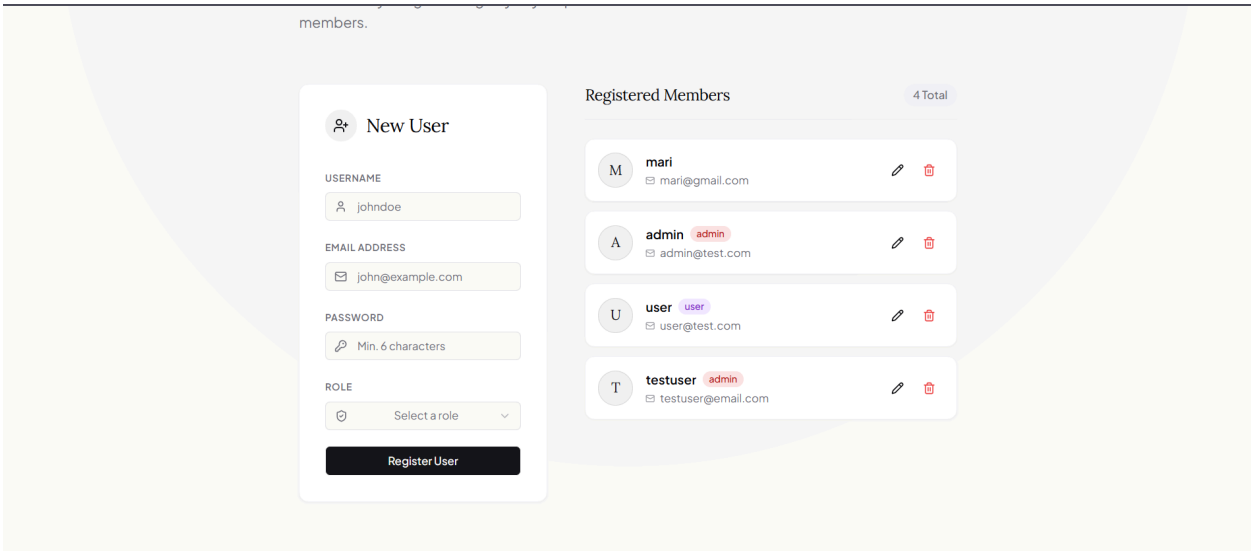
Login:



Successful Login/ Dashboard



Add a new user (edit and delete icons are included)



User restriction (regular user attempts to log into admin area)